

Warehouse Planning: Greedy Baseline and Enhanced Heuristic

Hazem & Yahya

Highlights

Problem Summary. We construct a feasible next-day plan for warehouse operations. The plan assigns orders to pickers, batches orders into trips, routes each batch through shelves, schedules restocks, and respects deadlines, shift limits, capacity, stock constraints, and perishable handling limits. We target feasibility and reasonable priority handling rather than optimality.

Approach Overview. We provide two greedy strategies:

- **Baseline Greedy:** orders sorted only by earliest deadline; restocking is triggered only when stock runs out. This gives a simple, interpretable reference.
- **Enhanced Greedy:** orders are scored by a weighted linear function of normalized features (priority, slack, stock urgency, expected handling time, and price), then sorted by score (deadline as tie-break). Restocking is scheduled with lead time so items arrive by the first time they are needed, including optional delay buffers for risky items.

Batching uses a composite rule: starting from dispatch, we repeatedly add the feasible order maximizing score $-\lambda \cdot \text{normalizedDistance}$ with $\lambda = 0.2$. Routes inside each batch are built by a nearest-neighbor tour over required shelves.

Correctness (Feasibility) Argument. The algorithms maintain feasibility through explicit checks:

- **Stock feasibility:** A batch is built only if the required quantities are available at the planned start time. Restocks, when scheduled, increase stock only after their arrival time.
- **Capacity feasibility:** A candidate order is added to a batch only if total weight remains within the picker capacity.
- **Release and shift feasibility:** A batch start time is the maximum of picker availability and order release; a batch is rejected if its finish time exceeds the picker shift plus overtime limit.
- **Deadline feasibility:** A batch is rejected if its dispatch time exceeds any order deadline in that batch.
- **Perishable feasibility:** For perishable items, the elapsed time since the last restock arrival is checked against the maximum allowed wait.

Because each accepted batch satisfies all constraints, the resulting plan is feasible whenever the algorithm succeeds.

Industrial Considerations.

- **Robustness to delays:** The enhanced method adds buffer time for high-risk restocks, improving deadline compliance in practice.
- **Operational transparency:** The baseline method is easy to explain; the enhanced method remains interpretable via its weighted score.
- **Scalability:** The greedy approach scales to large order sets and can be executed daily within typical operational time windows.
- **Safety and quality:** Perishable and fragile handling time constraints reduce spoilage and damage risk.

Testing Strategy. We test on multiple synthetic cases with increasing scale and difficulty. The cases include: (i) ample stock, (ii) understocked high-demand items requiring restock, (iii) high delay risk requiring buffer, and (iv) an infeasible VIP deadline to validate failure reporting. We compare baseline vs. enhanced outputs for feasibility and lateness differences.

Complexity (High Level). Let V be the number of nodes, M the number of orders, P the number of pickers, and S the number of shelves per batch.

- Floyd–Warshall for all-pairs shortest paths: $O(V^3)$.
- Sorting orders: $O(M \log M)$.
- Batch construction with score-distance selection: $O(M^2)$.
- Routing per batch (nearest neighbor): $O(S^2)$ per batch.

Overall, the dominant term is $O(V^3)$ for path preprocessing, followed by order processing and batching.

Appendix

A Examples and Edge Cases

Example A: Sufficient Stock. All orders can be scheduled without restock. Baseline and enhanced produce feasible plans; enhanced tends to group higher-priority orders earlier.

Example B: Understocked Item with Reserve. Baseline triggers restock only after stockout, which can delay the first affected orders. Enhanced schedules restock in advance based on lead time, reducing deadline misses.

Example C: High Delay Risk. An item with a large delay buffer causes late arrivals if not planned. Enhanced uses the buffer to avoid optimistic schedules; baseline often fails with late dispatch.

Example D: Infeasible Deadline. A VIP order requires an item whose restock arrival is later than its deadline. Both algorithms report infeasible with a clear reason.

B Test Results Summary

Table 1 summarizes runs for the four built-in cases. Runtime is the total for producing both baseline and enhanced plans in one run.

Case	Baseline Feasible	Enhanced Feasible	Baseline Restocks	Enhanced Restocks	Runtime (ms)
1	Yes	Yes	0	0	18.58
2	Yes	Yes	3	2	13.58
3	No (unassigned)	Yes	0	8	39.13
4	No (VIP deadline)	No (VIP deadline)	0	0	4.60

Table 1: Test results for cases 1–4.

C Baseline Greedy Pseudocode (Deadline-Only)

```
Sort orders by earliest deadline.
For each picker, track available_time.
For each order in order list:
    choose earliest-available picker
    start_time = max(picker.available_time, order.release)
    if stock insufficient:
        place restock request with creation_time = start_time
        wait until arrival
    build a batch by adding feasible orders maximizing
    score - 0.2 * normalizedDistance until capacity or constraints fail
    compute route via nearest-neighbor shelves
    if finish_time exceeds shift or deadline, report infeasible
    update stock and picker availability
```

after all orders: if any unassigned, infeasible

actions for restock:
 only triggered when stock is insufficient at start_time

deadlines checked per order in each batch

times and routes computed using all-pairs shortest paths

D Enhanced Greedy Pseudocode (Score + Lead-Time Restock)

Compute raw features per order: priorityWeight, slack, stockUrgency, expectedHandle, orderPrice.
 Normalize features across all orders.
 Score = $w_1 \text{priority} + w_2 \text{price} + w_3 \text{urgency} - w_4 \text{handle} - w_5 \text{slack}$.
 Sort by score desc; deadline is tie-break.

Pre-schedule restocks based on demand shortfall and lead time.
 If an order needs stock earlier than arrival, shift start_time to arrival.
 Apply delay buffer if delay_risk is high.

Batching uses score-distance selection with capacity and feasibility checks.
 Routes computed by nearest-neighbor tour over shelves.
 Feasibility checks: stock, perishable max_wait, capacity, shift, deadline.

E Correctness Sketch (More Detail)

Each accepted batch respects stock availability at the batch start time (plus restocks applied at or before that time). By checking perishable wait against the last restock arrival and verifying capacity, shift, and deadline bounds, the algorithm only accepts batches that satisfy constraints. Therefore, if the algorithm outputs a plan, it is feasible by construction.

F Complexity and Space

Let V be nodes, E edges, M orders, P pickers, B batches, and S shelves per batch.

- Time: Floyd–Warshall $O(V^3)$; sorting $O(M \log M)$; batch selection worst-case $O(M^2)$; routing $O(BS^2)$. In practice, S is small and batching is near-linear for typical constraints.
- Space: $O(V^2)$ for distances, $O(M)$ for orders, and $O(BS)$ for batch routes.

G Industrial Considerations (Extended)

- **Data quality:** Poor estimates of handling times or delay risk reduce schedule accuracy. Periodic calibration from logs is recommended.
- **Operational fairness:** Picker workloads can be balanced by rotating picker selection or adding workload penalties.

- **Robustness:** Restock buffers and conservative feasibility checks reduce the probability of late delivery at the cost of more restocks.
- **Explainability:** The enhanced score remains interpretable because each term reflects a tangible business factor.

H Development Notes and Time Log

Process Note. After defining the problem and splitting the work across subteams, we found the original problem harder than expected. We made simplifying assumptions and adjusted constraints to keep progress, but later realized we were drifting toward a different problem. We kept the core greedy algorithm and invested effort to adapt it back to the original constraints and assumptions.

Time Log (Approximate).

- 4 hours on the first draft of the enhanced solution (first hour focused on understanding the problem and building a baseline).
- 6 hours writing the pseudocode and adapting it to the original problem; this alignment step was the main bottleneck.